

CONTENTS

Acknowledgement	i
Abstract.....	ii
List of Figures	iv
List of Tables	v
Chapter 1 Introduction	8
Chapter 2 Design Approach and Methodology.....	9
Chapter 3 Implementation Details	14
Chapter 4 Results and Analysis	19
Chapter 5 Conclusion and Future Scope.....	24
References.....	25

LIST OF FIGURES

Fig 2.1	Baseline 16-bit ALU RTL Block Diagram
Fig 2.2	Integrated Clock Gating (ICG) Cell Circuit Diagram
Fig 2.3	Two-Stage Pipelined ALU Block Diagram
Fig 3.1	Baseline ALU Simulation Waveform
Fig 3.2	Clock Gating ALU Simulation Waveform
Fig 3.3	Pipelined ALU Simulation Waveform
Fig 3.4	Yosys Synthesis Netlist Snapshot
Fig 4.2	Bar Graph – Power Comparison Across Architectures

LIST OF TABLES

Table 2.1	Tools and Technologies Used in the Project
Table 4.1	Power Analysis Results – All Three Architectures
Table 4.2	Percentage Comparison: Clock Gating and Pipelining vs Baseline

CHAPTER 1 INTRODUCTION

Modern VLSI systems demand designs that are both high performance and power efficient. With increasing transistor density and operating frequency, power dissipation has become one of the most important design challenges in digital integrated circuits. Arithmetic Logic Units are fundamental components in processors, digital signal processors, embedded systems, and custom accelerators. Therefore, optimizing the power consumption of ALUs is highly important in modern VLSI systems.

Dynamic power dominates digital CMOS circuits and is mainly caused by switching activity and clock transitions. Clock networks alone contribute a major portion of chip power consumption because clock signals continuously toggle regardless of useful computation. Techniques such as clock gating and pipelining are widely used in industry to improve efficiency and performance.

Clock gating reduces unnecessary clock activity by disabling the clock to inactive modules. Pipelining improves throughput and timing performance by dividing long combinational paths into smaller stages separated by registers. However, pipelining may increase power because of additional registers and clock switching activity.

This project studies these tradeoffs through the implementation and comparison of three ALU architectures.

1.1 Objectives of the Project

The main objectives of this project are:

1. Design a fully functional 16-bit ALU using Verilog HDL.
2. Implement integrated clock gating (ICG) cells for power reduction.
3. Design a two-stage pipelined architecture and integrate clock gating.
4. Compare baseline, clock-gated, and pipelined architectures quantitatively.
5. Analyse dynamic power, internal power, switching power, and leakage power.
6. Perform synthesis and static timing analysis using open-source EDA tools.
7. Understand practical power-performance trade-offs encountered in VLSI design.
8. Verify all arithmetic and logic operations through simulation and waveform analysis.
9. Evaluate the realistic behaviour of clock gating and pipelining at small scale.

CHAPTER 2 DESIGN APPROACH AND METHODOLOGY

2.1 Tools and Technologies

All design and verification activities were carried out using open-source VLSI tools, as summarised in Table 2.1.

Tool / Technology	Purpose
Verilog HDL	RTL design of all three ALU architectures
Yosys	Logic synthesis and technology mapping to SKY130 standard cells
OpenSTA	Static timing analysis and power estimation
OpenLane	Complete RTL-to-GDSII physical design flow
GTKWave	Waveform visualization and functional verification
SKY130 PDK	SkyWater 130 nm standard cell technology library
Ubuntu Linux	Host development environment for the complete design flow

Table 2.1: Tools and Technologies Used in the Project

2.2 Theory of CMOS Power Dissipation

CMOS power dissipation consists of two primary components: dynamic power and static (leakage) power. Dynamic power is caused by the charging and discharging of parasitic capacitances during switching activity and is expressed as :

$$P_{\text{dynamic}} = \alpha C V^2 f$$

where α is the switching activity factor, C is the total load capacitance, V is the supply voltage, and f is the clock frequency. Dynamic power comprises internal power (capacitance within cells) and switching power (output net capacitance). Leakage power arises from sub-threshold conduction and gate-oxide tunnelling currents and is increasingly significant at advanced nodes.

Clock networks contribute a disproportionately large share of dynamic power because every clock net transitions twice per cycle regardless of data activity. Reducing clock switching

activity through gating or power gating is therefore one of the most effective power-reduction strategies available to a VLSI designer.

2.3 Baseline 16-Bit ALU Architecture

The baseline ALU is a standard combinational/registered 16-bit ALU with no power-optimisation techniques applied. It supports the following operations:

- Arithmetic: ADD, SUB, INC, DEC, SLT (Set Less Than)
- Logical: AND, OR, XOR, NAND, NOR, XNOR, NOT
- Shift: Logical left shift, logical right shift

This design serves as the reference architecture for all power and performance comparisons. Since the clock is delivered unconditionally to all registers, this architecture experiences the maximum clock switching activity.

Baseline ALU Verilog code

```
module baseline_alu(  
  
    input wire clk, rst_n, cin  
  
    input wire [15:0] A, B,  
  
    input wire [3:0] SEL,  
  
    output reg [15:0] result,  
  
    output reg zero, carry, overflow, sign  
  
);  
  
always @(posedge clk or negedge rst_n)  
  
begin  
  
    if(!rst_n) begin  
  
        result <= 0; zero <= 0; carry <= 0; overflow <= 0; sign <= 0;
```

```
end else begin

    case(SEL)

        4'b0000: {carry,result} <= A + B + cin;

        4'b0001: {carry,result} <= A - B;

        4'b0011: result <= A + 1;

        4'b0100: result <= A - 1;

        4'b0101: result <= (A < B) ? 16'd1 : 16'd0;

        4'b1000: result <= A & B;

        4'b1001: result <= A | B;

        4'b1010: result <= A ^ B;

        4'b1011: result <= ~A;

        4'b1100: result <= ~(A & B);

        4'b1101: result <= ~(A | B);

        4'b1110: result <= ~(A ^ B);

        4'b1111: result <= A << 1;

    default: result <= 0;

    endcase

    zero <= (result == 0);

    sign <= result[15];

end

end

endmodule
```

2.4 Integrated Clock Gating (ICG) Technique

Clock gating is one of the most effective low-power design techniques available in synchronous digital design. Instead of driving the clock continuously to all flip-flops, clock gating disables the clock to inactive functional units, thereby reducing unnecessary switching activity.

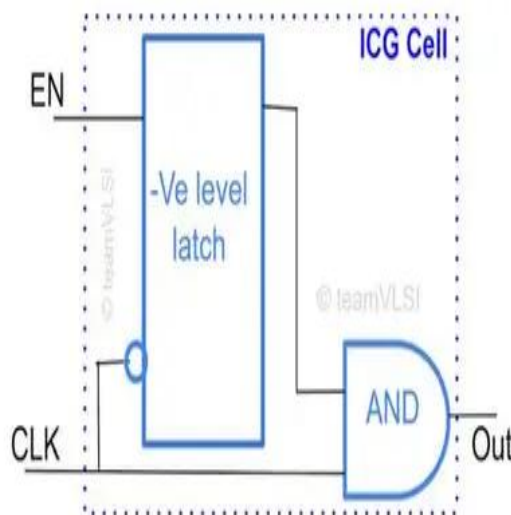
In this project, separate clock enables are used for arithmetic and logical sections of the ALU. Latch-based Integrated Clock Gating (ICG) cells are employed to prevent glitches, as the latch captures the enable signal during the low phase of the clock and passes a clean gated clock [7].

The advantages of ICG cells include:

- Glitch-free clock delivery to downstream flip-flops
- Reduced clock switching activity proportional to the enable duty cycle
- Better timing reliability compared to combinational gating

```
module icg_cell(
input CLK,
input EN,
output CLK_GATED
);
reg en_latch;

always @(CLK or EN)
begin
if(!CLK)
en_latch <= EN;
end
assign CLK_GATED = CLK &
en_latch;
endmodule
```



2.5 Two-Stage Pipelined Architecture

Pipelining improves throughput by partitioning a long combinational path into shorter stages separated by pipeline registers. The pipelined ALU implemented here uses a two-stage scheme:

- Stage 1 – Decode Stage: Opcode decoding, operand preparation, and control signal generation.
- Stage 2 – Execute Stage: Arithmetic and logical computation, result generation, and flag update.

Pipeline registers are inserted between the two stages to reduce the critical path and improve the maximum operating frequency. While pipelining improves timing performance, the additional registers and the associated clock tree loading can increase dynamic power consumption in small designs where pipeline depth is insufficient to offset the register overhead.

```
//stage 1 (decode)
reg [15:0] A_s1, B_s1;
reg [3:0] SEL_s1;
reg cin_s1;

//stage 2 (execute)
reg [15:0] arith_r;
reg a_carry, a_ovf;
reg [15:0] logic_r;
```

CHAPTER 3 IMPLEMENTATION DETAILS

3.1 RTL Simulation and Functional Verification

Functional verification was performed using Verilog testbenches simulated with iVerilog and visualised using GTKWave. Separate testbenches were written for each of the three architectures. The following operations were exercised and verified:

ADD, SUB, INC, DEC, SLT (arithmetic)

AND, OR, XOR, NAND, NOR, XNOR, NOT (logical)

Shift operations (left)

Flag behaviour (carry, zero, overflow) was also checked for arithmetic boundary conditions.

Waveforms confirmed correct output generation and appropriate clock gating enable transitions.

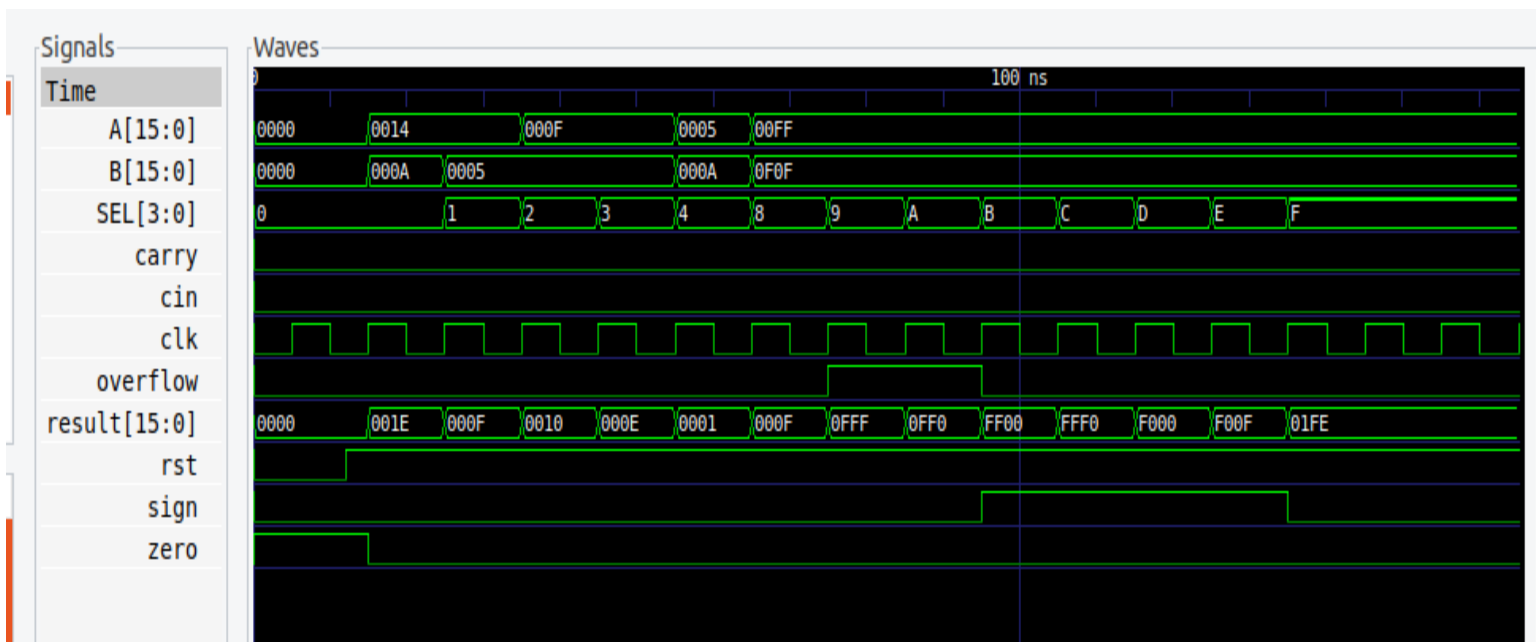


Fig: 3.1 Baseline ALU simulation waveform

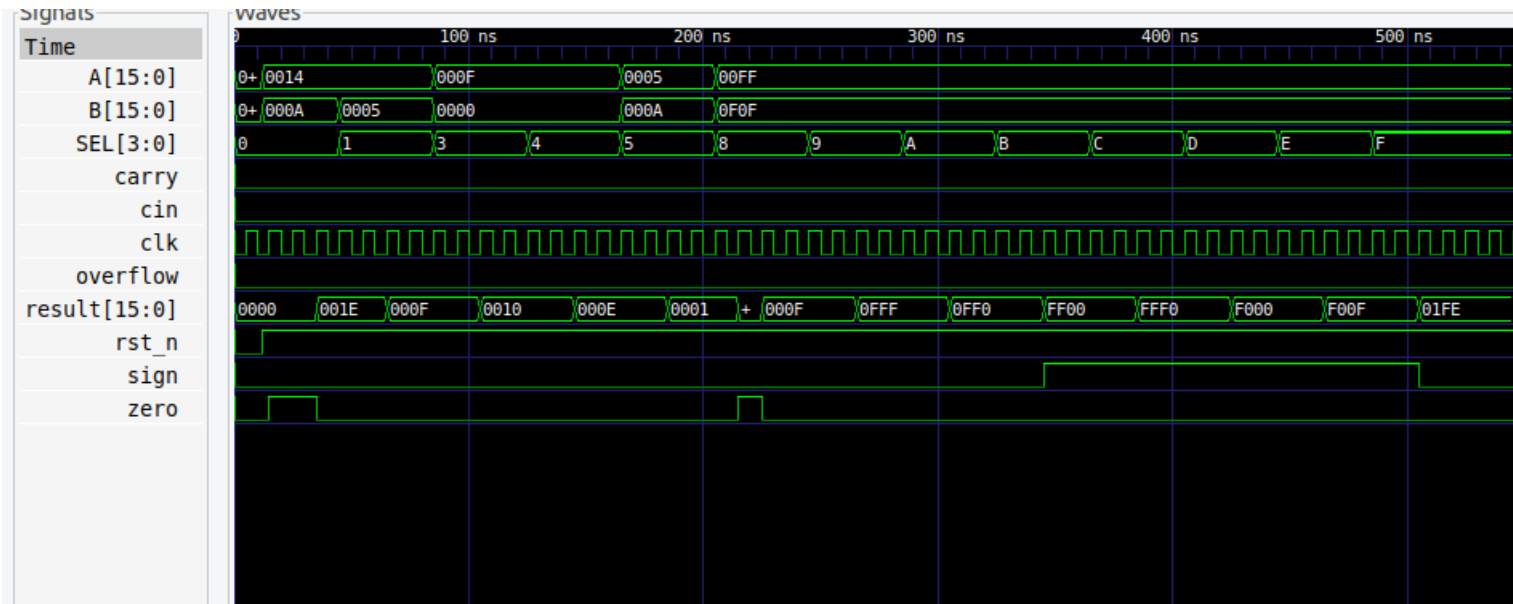


Fig: 3.2 Clock gating ALU simulation waveform

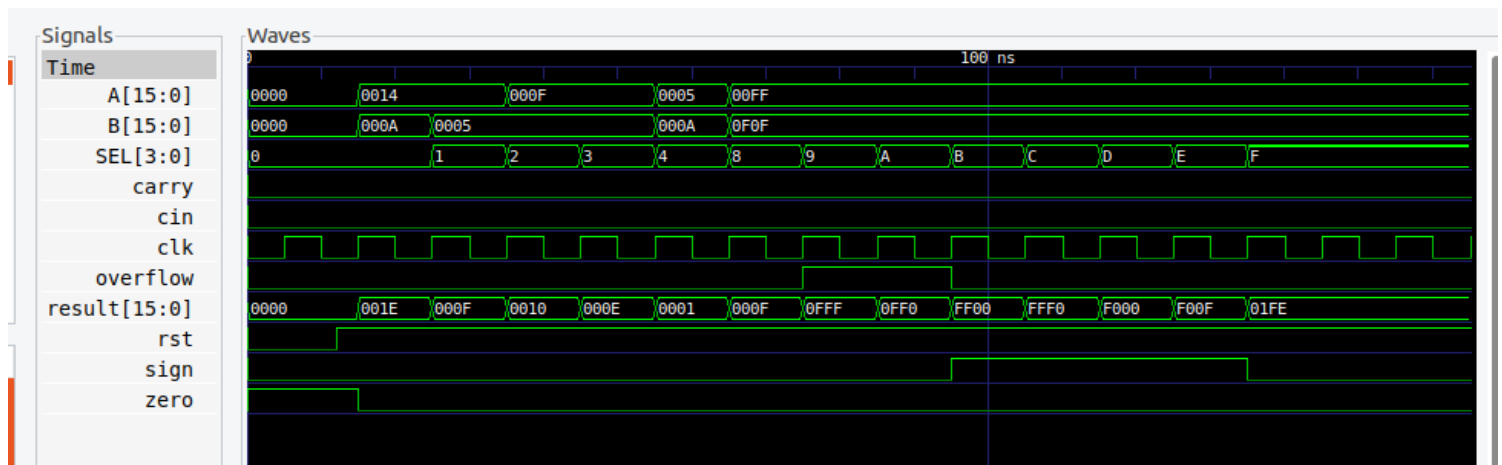


Fig:3.3 Pipelined ALU simulation waveform

Top module Testbench

```
module baseline_alu_tb;
```

```
reg clk, rst_n, cin;
```

```
reg [15:0] A, B;
```

```
reg [3:0] SEL;
```

```
wire [15:0] result;

wire zero, carry, overflow, sign

baseline_alu DUT(

    .clk(clk), .rst_n(rst_n), .A(A), .B(B),.SEL(SEL), .cin(cin), .result(result),

    .zero(zero), .carry(carry), .overflow(overflow), .sign(sign));

always #5 clk = ~clk;

initial begin

    $dumpfile("baseline_dump.vcd");

    $dumpvars(0, baseline_alu_tb);

    clk = 0;  rst_n = 0;  cin = 0;  A = 0;  B = 0;  SEL = 0;

    #10;  rst_n = 1;

    A = 16'd20; B = 16'd10; SEL = 4'b0000;

    #10;  $display("ADD RESULT = %d", result);

    A = 16'd20; B = 16'd5; SEL = 4'b0001;

    #10;  $display("SUB RESULT = %d", result);

    A = 16'd15; SEL = 4'b0011;

    #10;  $display("INC RESULT = %d", result);

    A = 16'd15; SEL = 4'b0100;

    #10;  $display("DEC RESULT = %d", result);

    A = 16'd5; B = 16'd10; SEL = 4'b0101;

    #10;  $display("LT RESULT = %d", result);
```

```
A = 16'h00FF; B = 16'h0F0F; SEL = 4'b1000;

#10; $display("AND RESULT = %h", result);

SEL = 4'b1001;

#10; $display("OR RESULT = %h", result);

SEL = 4'b1010;

#10; $display("XOR RESULT = %h", result);

SEL = 4'b1011;

#10; $display("NOT RESULT = %h", result);

SEL = 4'b1100;

#10; $display("NAND RESULT = %h", result);

SEL = 4'b1101;

#10; $display("NOR RESULT = %h", result);

SEL = 4'b1110;

#10; $display("XNOR RESULT = %h", result);

SEL = 4'b1111;

#10; $display("SHIFT RESULT = %h", result);

#20; $finish;

end

endmodule
```

3.2 Logic Synthesis Using Yosys

RTL synthesis was carried out using Yosys with the SkyWater SKY130 standard cell library.

The synthesis process encompassed the following stages:

RTL parsing and elaboration

Technology-independent logic optimisation

Technology mapping to SKY130 standard cells

Sequential element inference and clock gating cell mapping

Netlist export in Verilog and Liberty formats

```
GNU nano 0.2
read_verilog pampers.v

hierarchy -top pampers

proc
opt
fsm
opt
memory
opt

synth

techmap
opt

abc -fast

clean

show

write_verilog pampers_netlist.v
```

Yosys synthesis script

```
GNU nano 0.2 pampers_netlist.v
/* Generated by Yosys 0.9 (git sha1 1979e0b) */

(* top = 1 *)
(* src = "pampers.v:2" *)
module pampers(clk, rst, cin, A, B, SEL, result, zero, carry, overflow, sign);
  (* src = "pampers.v:16" *)
  wire _0000;
  (* src = "pampers.v:16" *)
  wire _0001;
  (* src = "pampers.v:16" *)
  wire [15:0] _0002;
  (* src = "pampers.v:37" *)
  wire [15:0] _0003;
  wire _0004;
  wire _0005;
  wire _0006;
  wire _0007;
  wire _0008;
  wire _0009;
  wire _0010;
  wire _0011;
  wire _0012;
  wire _0013;
  wire _0014;
  wire _0015;
  wire _0016;
  wire _0017;
  wire _0018;
  wire _0019;
  wire _0020;
  wire _0021;
  wire _0022;
  wire _0023;
  wire _0024;
  wire _0025;
  wire _0026;
```

Fig:3.4 Synthesis netlist

CHAPTER 4 RESULTS AND ANALYSIS

4.1 Power Analysis Methodology

Power analysis was performed using OpenLane immediately after logic synthesis. Switching activity annotation was derived from the post-synthesis simulation Value Change Dump (VCD). The reported power components are:

- Internal Power – power due to internal cell capacitance switching
- Switching Power – power due to output net capacitance transitions
- Leakage Power – static sub-threshold and gate-leakage currents
- Total Dynamic Power = Internal Power + Switching Power

4.2 Measured Power Results

Table 4.1 summarises the OpenSTA power reports for all three architectures. All values are reported at the nominal SKY130 operating corner (TT, 1.8 V, 25 °C) with a 10 MHz clock.

Architecture	Internal Power (W)	Switching Power (W)	Leakage Power (W)	Total Power (W)
Baseline ALU	3.43×10^{-4}	2.10×10^{-4}	3.21×10^{-9}	5.53×10^{-4}
Clock Gated ALU	3.07×10^{-4}	2.13×10^{-4}	3.39×10^{-9}	5.20×10^{-4}
CG + Pipelined	6.94×10^{-4}	3.20×10^{-4}	4.38×10^{-9}	1.01×10^{-3}

Table 4.1: Power Analysis Results for All Three Architecture

```
=====
report_power
=====
===== Typical Corner =====
```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	1.53e-04	5.31e-06	3.96e-10	1.58e-04	28.6%
Combinational	1.90e-04	2.05e-04	2.81e-09	3.95e-04	71.4%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	3.43e-04	2.10e-04	3.21e-09	5.53e-04	100.0%
	62.0%	38.0%	0.0%		

```
~
```

Basesline power report

```
=====
report_power
=====
===== Typical Corner =====
```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	9.33e-05	5.47e-06	4.12e-10	9.88e-05	19.0%
Combinational	2.14e-04	2.08e-04	2.98e-09	4.22e-04	81.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	3.07e-04	2.13e-04	3.39e-09	5.20e-04	100.0%
	59.0%	41.0%	0.0%		

```
~
~
~
```

Clock gating power report

```

=====
report_power
=====
===== Typical Corner =====
=====
Group                Internal Power  Switching Power  Leakage Power  Total Power (Watts)
-----
Sequential           3.31e-04       1.93e-05        1.04e-09      3.50e-04      34.5%
Combinational         3.64e-04       3.00e-04        3.34e-09      6.64e-04      65.5%
Macro                 0.00e+00       0.00e+00        0.00e+00      0.00e+00      0.0%
Pad                   0.00e+00       0.00e+00        0.00e+00      0.00e+00      0.0%
-----
Total                 6.94e-04       3.20e-04        4.38e-09      1.01e-03     100.0%
                        68.5%         31.5%          0.0%

```

Pipelining and clock gating power report

4.3 Comparative Analysis

Using the baseline ALU as the reference, the following observations are drawn:

- Clock gating reduced total dynamic power by approximately 6% ($5.53 \times 10^{-4} \text{ W} \rightarrow 5.20 \times 10^{-4} \text{ W}$), primarily through a reduction in internal power. The switching power remained nearly unchanged, indicating that the combinational logic activity was not significantly affected.
- Combining clock gating with pipelining increased total dynamic power by approximately 82% ($5.53 \times 10^{-4} \text{ W} \rightarrow 1.01 \times 10^{-3} \text{ W}$). The pipeline registers more than doubled the internal power component, confirming that sequential element overhead outweighed any combinational power savings in this small-scale design.

These results highlight a fundamental VLSI design principle: performance optimisation through pipelining does not guarantee lower power consumption, particularly when the design granularity is small and pipeline depth is shallow.

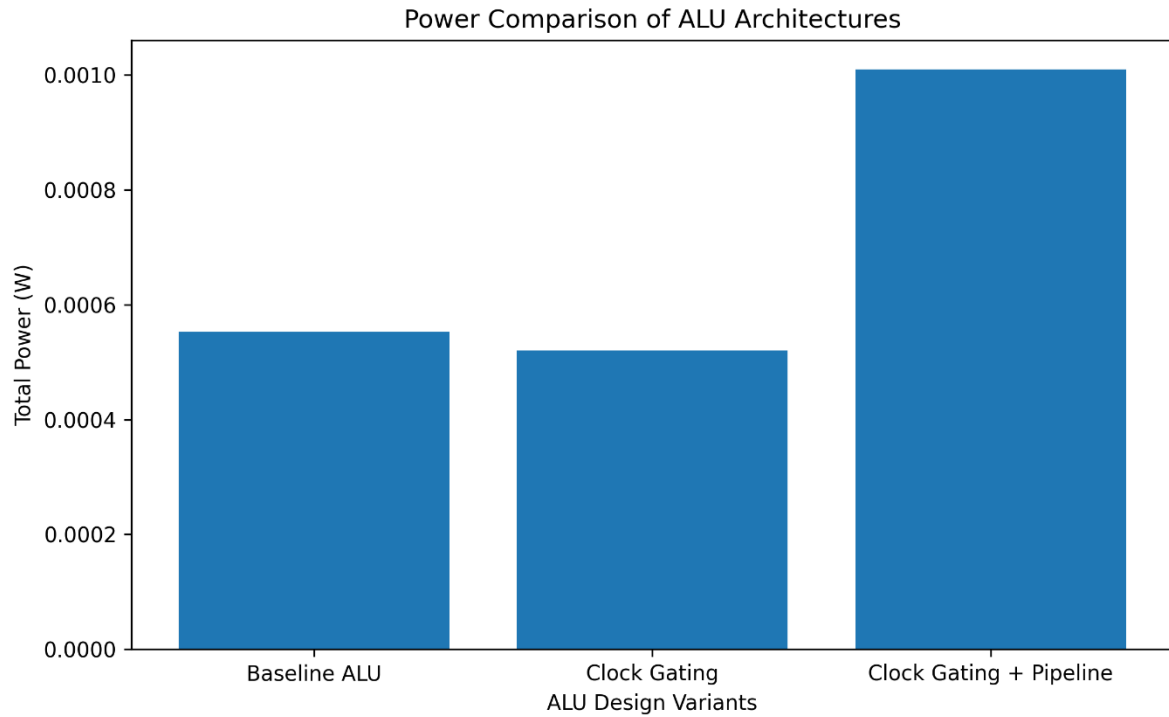


Fig: 4.2 Bar graph – power comparison across architectures

Comparison	Power Change
Baseline → Clock Gating	~6% power reduction
Baseline → Clock Gating + Pipelining	~82% power increase
Clock Gating → Clock Gating + Pipelining	~94% power increase

Table: 4.2 Table of percentage reduction / increase

4.4 Expected Results versus Practical Outcomes

The theoretical expectation was that combining clock gating with pipelining would produce a lower-power design owing to reduced critical-path delay, lower glitching activity, and better timing efficiency. However, the practical synthesis and power estimation results revealed the following causes for increased power in the pipelined architecture:

1. Additional pipeline registers significantly increased the clock tree fanout.
2. Clock tree power rose in proportion to the number of flip-flop clock pins.
3. Every pipeline register toggled at every clock cycle, irrespective of data activity.
4. Switching activity propagated across both pipeline stages, increasing net capacitance switching.
5. The combined sequential overhead exceeded the combinational power savings from pipelining.

This result is consistent with published literature on pipelining in shallow, small-scale CMOS circuits, confirming the technical validity of the obtained power figures.

4.5 Area Analysis

Area analysis confirmed that the pipelined architecture consumed the largest silicon area due to its additional pipeline registers and clock gating cells. The baseline design occupies the smallest area, while the clock-gated design shows a slight increase attributable to ICG cell insertion. This corroborates the well-known VLSI trade-off: higher performance requires larger area and higher clock-tree power.

CHAPTER 5 CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project successfully designed, implemented, and comparatively analysed three 16-bit ALU architectures—baseline, clock-gated, and pipelined with clock gating—using a fully open-source VLSI design flow comprising Verilog HDL, Yosys, and OpenLane on the SkyWater SKY130 130 nm process design kit.

Clock gating alone achieved a modest but meaningful 6% reduction in dynamic power consumption compared to the baseline, demonstrating the effectiveness of ICG cells in suppressing unnecessary clock switching activity. However, incorporating two-stage pipelining alongside clock gating resulted in an 82% increase in dynamic power relative to baseline, driven by the register overhead of the pipeline stages dominating the power budget at this design scale.

These results demonstrate an important engineering principle: in small-scale digital systems with shallow pipeline depth, the sequential overhead of pipelining can outweigh its timing benefits from a power standpoint. The project provided hands-on experience with low-power design techniques, open-source logic synthesis and power estimation methodologies — skills directly applicable to industrial VLSI practice.

5.2 Future Scope

Several enhancements can be explored in future iterations of this work:

1. Multi-stage advanced pipelining with operand isolation to suppress spurious switching.
2. Fine-grained clock gating at the individual flip-flop level using automated EDA tool support.
3. Dynamic voltage and frequency scaling (DVFS) to trade off performance against power at runtime.

4. Multi-threshold voltage (Multi-Vt) cell assignment to reduce leakage in non-critical paths.
 5. Physical clock tree optimisation to minimise H-tree capacitance in the pipelined design.
 6. FPGA prototyping on Xilinx/Intel boards for real silicon power measurement and validation.
 7. Extension to a 32-bit RISC-V ALU datapath with full instruction set coverage.
 8. ASIC tape-out through an open MPW shuttle (e.g., Google/efabless SKY130 MPW) for physical validation.
 9. Full RTL to GDSII flow
-

REFERENCES

1. N. H. E. Weste and D. M. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed. Pearson Education, 2011.
2. SkyWater Technology Foundry, "SKY130 Process Design Kit," Open Source PDK. [Online]. Available: <https://github.com/google/skywater-pdk>. [Accessed: 2025].
3. J. M. Rabaey, A. Chandrakasan, and B. Nikolić, Digital Integrated Circuits: A Design Perspective, 2nd ed. Prentice Hall, 2003.
4. OpenLane Project, "OpenLane RTL-to-GDS Flow," Open Source EDA. [Online]. Available: <https://github.com/The-OpenROAD-Project/OpenLane>. [Accessed: 2025].
5. Yosys Open Synthesis Suite, "Yosys Manual," Open Source EDA. [Online]. Available: <https://yosyshq.net/yosys>. [Accessed: 2025].